

VIETNAM NATIONAL UNIVERSITY HCMC
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE & ENGINEERING



LAB 1 REPORT

Student Information – Statistical Learning

Course: IT171IU

Lab: Lab 1: Statistical Learning & Introduction to Python

Student's Name	StudentID	Class/Lab group	Date
Nguyễn Anh Tú	ITDSIU23026	IT171IU/Group 1	3/10/2026

TABLE OF CONTENTS

Part 0: Environment Setup (.venv).....	3
Part 1: Goals and Objectives.....	4
Part 2: Theory.....	4
Part 3: Exercise Analysis	5
Exercise 1 - NumPy warm-up	5
Exercise 2 - Random numbers, correlation and a saved figure.....	7
Exercise 3 - Cleaning the Auto data	8
Exercise 4 - Subsetting rows with iloc[].....	10
Exercise 5 - Graphical exploration	10
Exercise 6 - The College data set	12
Exercise 7 - Loops and string formatting.....	15
Exercise 8 - Estimating bias and variance by simulation	16
Exercise 9 - Choosing K for KNN	16
Part 5: Scholar Discussion	17
Part 6: Final Reflection.....	18

PART 0: ENVIRONMENT SETUP (.VENV)

– Task Requirement

Set up an isolated virtual environment (.venv) for the course and install necessary packages to ensure code reproducibility and avoid dependency conflicts.

– Step-by-Step Implementation

- Step 1: Created a root folder named IT171IU-labs and navigated into it via the terminal.
- Step 2: Executed `python -m venv .venv` to create the virtual environment.
- Step 3: Activated the environment using `.venv\Scripts\activate` (Windows).
- Step 4: Installed core packages by executing `pip install ISLP jupyterlab notebook ipykernel`.
- Step 5: Registered the environment as a Jupyter kernel named "IT171IU" to ensure VS Code/Jupyter utilizes the correct interpreter.

– Execution Trace & Output[Insert screenshot of your Task 0 Environment Check terminal output here]



```
# [ADDED] worked example - run this cell
# Environment check - run in each environment and include the output in your report
import sys, platform, importlib
print("Python version :", sys.version.split()[0])
print("Executable      :", sys.executable)
print("Operating sys.  :", platform.system(), platform.release())
for pkg in ["numpy", "pandas", "matplotlib", "sklearn", "statsmodels", "ISLP"]:
    try:
        m = importlib.import_module(pkg)
        print(f"{pkg:<12} {getattr(m, '__version__', 'installed')}")
    except ImportError:
        print(f"{pkg:<12} NOT INSTALLED [ ] run the Setup cell / pip install ISLP")
```

[19] ✓ 0.0s Python

```
Python version : 3.14.7
Executable      : d:\DOWNLOADS\Statistical learning\IT171IU_Lab1_Nguyễn Anh Tú_ITDSIU23026\.venv\Scripts\python.exe
Operating sys.  : Windows 10
numpy           2.5.3
pandas          2.3.3
matplotlib      3.11.2
sklearn         1.9.1
statsmodels     0.15.0
ISLP            0.4.1
```

```
# 🍌 Task 0 - environment check in two environments - your code
# Write your code below (replace the TODOs), then run the cell (Shift+Enter).
full_name = "Nguyen Anh Tú" # <- replace with your full name
print(f"Hello, {full_name}! My statistical-learning environment works.")

[20] ✓ 0.0s Python
... Hello, Nguyen Anh Tú! My statistical-learning environment works.
```

PART 1: GOALS AND OBJECTIVES

This lab aims to bridge foundational Python programming with the core mathematical paradigms of statistical learning. The key objectives addressed include:

- Understanding the statistical-learning model $Y = f(X) + \epsilon$ and distinguishing between prediction and inference.
- Analyzing the bias-variance trade-off and differentiating training error from test error.
- Creating and manipulating robust numerical data structures using NumPy arrays (shapes, slicing, element-wise operations).
- Loading, cleaning, and extracting statistical summaries from datasets using pandas DataFrames.
- Visualizing data distributions and relationships using Matplotlib.

PART 2: THEORY

The exercises in this lab heavily rely on the following theoretical concepts:

- *Statistical Learning Framework*: We assume a quantitative response Y relates to predictors X via an unknown function f plus an irreducible error ϵ . The expected prediction error is decomposed into reducible error (which we minimize) and irreducible error (variance of ϵ).
- *Model Accuracy & Bias-Variance Trade-off*: As model flexibility increases, training MSE strictly decreases. However, test MSE forms a U-shape, decomposing into Variance (how much $\hat{f}$ changes across datasets) and Bias (error introduced by approximating real-world complexity).

- *Data Structures & Indexing (Python Facts)*: Python relies on 0-based indexing where slices [start:stop] exclude the stop index. Additionally, slicing a NumPy array creates a 'view' (modifying it alters the original), necessitating `.copy()` for independent operations.
- *Reproducibility*: Random number generation requires a fixed seed (e.g., `np.random.default_rng(seed)`) to yield identical results across executions.

PART 3: EXERCISE ANALYSIS

Exercise 1 - NumPy warm-up

1. *Task Requirement*: Create a numeric vector, reshape it into a 4x5 matrix, extract specific subsets using slicing, compute dimensional statistics, and demonstrate the behavioral difference between an array view and a true copy.
2. *Concepts Used*: `np.arange()`, `.reshape()`, slicing [row, col], axis aggregation, `.copy()`.
3. Actual Code & Output

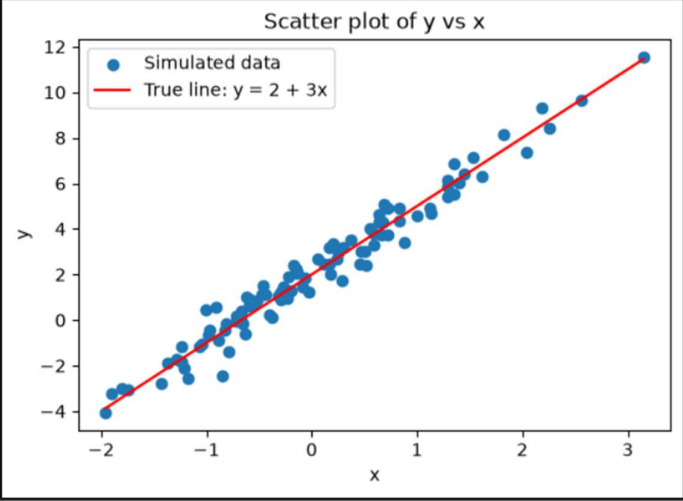
a/	<pre> import numpy as np # (a) # TODO (a): Create the vector with np.arange() # Vector v = (2, 4, 6, ..., 40) v = np.arange(2, 42, 2) # TODO (a): reshape it into a 4x5 matrix M M = v.reshape((4, 5)) print("Matrix M:\n", M) ✓ 0.0s Matrix M: [[2 4 6 8 10] [12 14 16 18 20] [22 24 26 28 30] [32 34 36 38 40]] </pre>
----	--

b/	<pre> if M is not None: print(M.shape, M.ndim, M.dtype) # (b) print(M.T) </pre> [25] ✓ 0.0s <pre> ... (4, 5) 2 int64 [[2 12 22 32] [4 14 24 34] [6 16 26 36] [8 18 28 38] [10 20 30 40]] </pre>	
c/	<pre> # (c) print("Second row:\n", M[1, :]) print("Last column:\n", M[:, -1]) print("Sub-matrix (rows 2-3, cols 1-3):\n", M[1:3, 0:3]) </pre> ✓ 0.0s <pre> Second row: [12 14 16 18 20] Last column: [10 20 30 40] Sub-matrix (rows 2-3, cols 1-3): [[12 14 16] [22 24 26]] </pre> Generate	
d/	<pre> # (d) print("Row means:", M.mean(axis=1)) print("Column standard deviations:", M.std(axis=0)) </pre> ✓ 0.0s <pre> Row means: [6. 16. 26. 36.] Column standard deviations: [11.18033989 11.18033989 11.18033989 11.18033989 11.18033989] </pre>	
e/	<pre> # (e) B = M[:2, :2] B[0, 0] = -1 print("Modified B:\n", M[0, 0]) M[0, 0] = 2 B_copy = M[:2, :2].copy() B_copy[0, 0] = -1 print("Copy of B (modified):\n", M[0, 0]) </pre> ✓ 0.0s <pre> Modified B: -1 Copy of B (modified): 2 </pre>	

Exercise 2 - Random numbers, correlation and a saved figure

1. *Task Requirement:* Simulate a linear relationship with added Gaussian noise, compute the correlation coefficient, plot the simulated data against the true theoretical line, and export the figure.
2. *Concepts Used:* `np.random.default_rng()`, `standard_normal()`, `np.corrcoef()`, Matplotlib `subplots()`.
3. *Actual Code & Output*

a/	<pre>import numpy as np from matplotlib.pyplot import subplots # (a) rng = np.random.default_rng(2026) x = rng.standard_normal(100) eps = 0.5 * rng.standard_normal(100)</pre> ✓ 0.0s
b/	<pre># (b) y = 2 + 3 * x + eps print("Correlation between x and y:", np.corrcoef(x, y)[0, 1])</pre> ✓ 0.0s Correlation between x and y: 0.9820330014880412

c/	<pre> # (C) from matplotlib.pyplot import subplots fig, ax = subplots(figsize=(6, 4)) ax.scatter(x, y, marker='o', label='Simulated data') x_line = np.linspace(x.min(), x.max(), 100) ax.plot(x_line, 2 + 3 * x_line, color='red', label="True line: y = 2 + 3x") ax.set_xlabel('x') ax.set_ylabel('y') ax.legend() ax.set_title('Scatter plot of y vs x') fig.savefig("ex2_scatter.png", dpi=150) </pre> ✓ 0.3s  The figure is a scatter plot titled "Scatter plot of y vs x". The x-axis ranges from -2 to 3, and the y-axis ranges from -4 to 12. Blue circular markers represent "Simulated data" points, which are scattered around a red line representing the "True line: y = 2 + 3x". The legend in the top-left corner identifies the blue dots as "Simulated data" and the red line as "True line: y = 2 + 3x".
d/	

Exercise 3 - Cleaning the Auto data

1. *Task Requirement*: Import a raw dataset, identify and remove missing values, map qualitative representations, and aggregate summary statistics for all quantitative columns.
2. *Concepts Used* Pandas read_csv, dropna(), .map(), .agg().

3. Actual Code & Output

a/

```
import pandas as pd
# (a)
Auto_raw = pd.read_csv("ISLP-datasets/Auto.csv", na_values=["?"])
total_rows = Auto_raw.shape[0]
print("Total rows in Auto_raw:", total_rows)
Auto_clean = Auto_raw.dropna()
print("Total rows after dropping missing values:", Auto_clean.shape[0])
n_missing = total_rows - Auto_clean.shape[0]
print("Number of missing values dropped:", n_missing)
```

✓ 0.0s

Total rows in Auto_raw: 392
Total rows after dropping missing values: 392
Number of missing values dropped: 0

b/

```
# (b)
print("Qualitative variables: origin, name")
print("Quantitative variables: mpg, cylinders, displacement, horsepower, weight, acceleration, year\n")

Auto_clean = Auto_clean.copy()
Auto_clean["origin"] = Auto_clean["origin"].map({1: "America", 2: "Europe", 3: "Japan"})

print(Auto_clean["origin"].head())
```

✓ 0.0s

Qualitative variables: origin, name
Quantitative variables: mpg, cylinders, displacement, horsepower, weight, acceleration, year

0	America
1	America
2	America
3	America
4	America

Name: origin, dtype: object

c/

```
# (c)
quantitative = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "year"]
stats_table = Auto_clean[quantitative].agg(["min", "max", "mean", "std"]).T
stats_table["range"] = stats_table["max"] - stats_table["min"]
stats_table = stats_table[["range", "mean", "std"]]
stats_table
```

✓ 0.0s

	range	mean	std
mpg	37.6	23.445918	7.805007
cylinders	5.0	5.471939	1.705783
displacement	387.0	194.411990	104.644004
horsepower	184.0	104.469388	38.491160
weight	3527.0	2977.584184	849.402560
acceleration	16.8	15.541327	2.758864
year	12.0	75.979592	3.683737

Exercise 4 - Subsetting rows with iloc[]

1. *Task Requirement:* Remove a specific continuous chunk of rows (10th through 85th) from the cleaned dataset and analyze how this structural deletion shifts the summary statistics.
2. *Concepts Used:* np.r_, Pandas .iloc[].
3. *Actual Code & Output*

```
keep = np.r_[0:9, 85:len(Auto_clean)]
Auto_sub = Auto_clean.iloc[keep]
summary_sub = Auto_sub[quantitative].agg(["min", "max", "mean", "std"]).T
summary_sub["range"] = summary_sub["max"] - summary_sub["min"]
print("Remove the 10th through 85th observations:")
print(summary_sub[["range", "mean", "std"]])
```

✓ 0.0s

Remove the 10th through 85th observations:

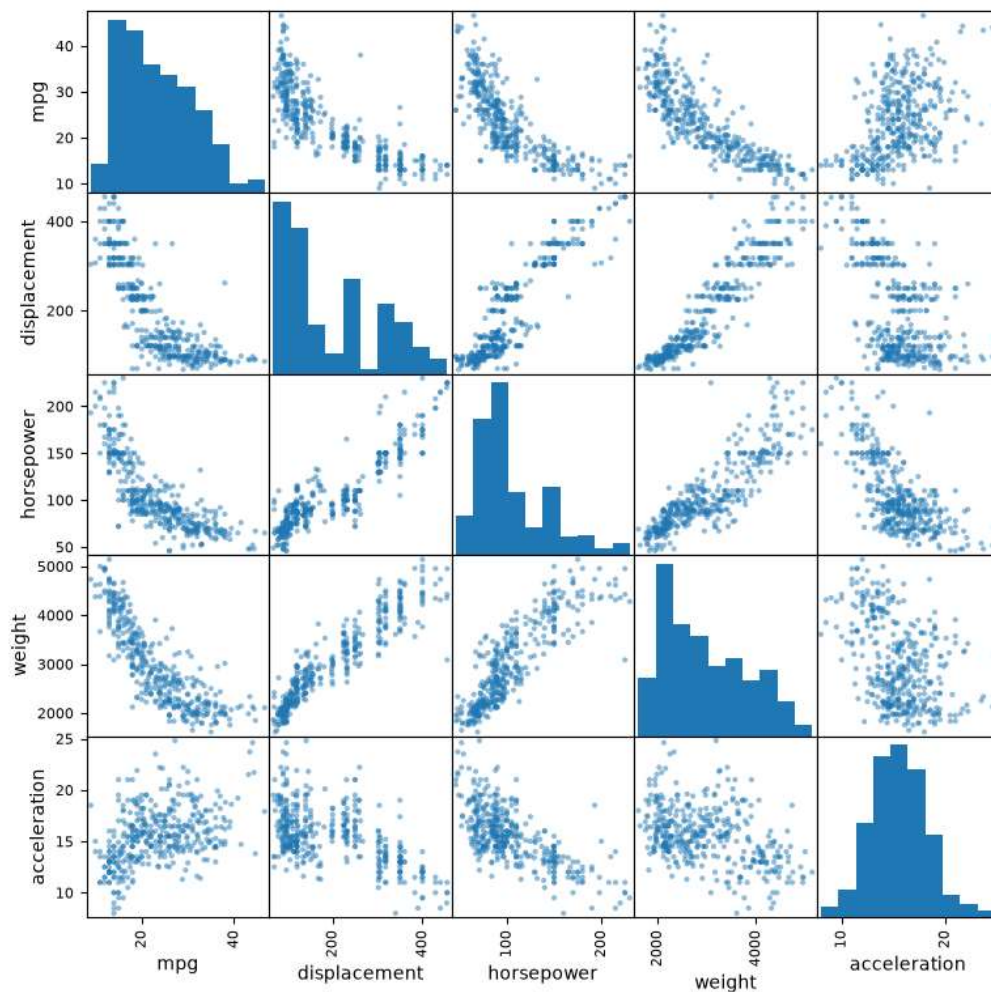
	range	mean	std
mpg	35.6	24.404430	7.867283
cylinders	5.0	5.373418	1.654179
displacement	387.0	187.240506	99.678367
horsepower	184.0	100.721519	35.708853
weight	3348.0	2935.971519	811.300208
acceleration	16.3	15.726899	2.693721
year	12.0	77.145570	3.106217

Exercise 5 - Graphical exploration

1. *Task Requirement:* Visualize multivariate correlations via a scatter-plot matrix and compare categorical distributions using grouped box plots to derive analytical findings.
2. *Concepts Used:* pd.plotting.scatter_matrix(), DataFrame.boxplot().
3. *Actual Code & Output*

```
a
/
#(a)
pd.plotting.scatter_matrix(
    Auto_clean[["mpg", "displacement", "horsepower", "weight", "acceleration"]],
    figsize=(9, 9)
)
```

✓ 1.1s



b

/

```
# (b)
fig, axes = subplots(1, 2, figsize=(12, 4))

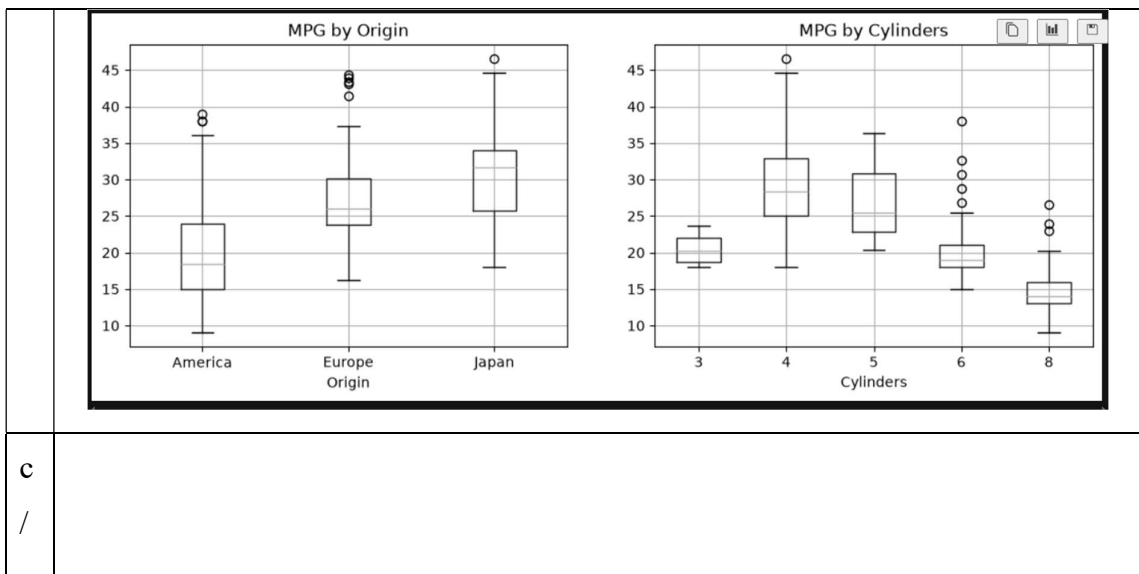
Auto_clean.boxplot("mpg", by="origin", ax=axes[0])
axes[0].set_title("MPG by Origin")
axes[0].set_xlabel("Origin")

Auto_clean.boxplot("mpg", by="cylinders", ax=axes[1])
axes[1].set_title("MPG by Cylinders")
axes[1].set_xlabel("Cylinders")

fig.suptitle("")

✓ 0.4s

Text(0.5, 0.98, '')
```



c
/

Exercise 6 - The College data set

1. *Task Requirement:* Load an external dataset, apply boolean masking to engineer a new categorical feature ('Elite'), and explore distributions via grouped boxplots and dynamic histograms.
2. *Concepts Used:* `load_data()`, boolean masking (`.loc`), `.hist()` with bins.
3. *Actual Code & Output*

a/

```
# (a)
from ISLP import load_data
College = load_data("College")
print("Dataset shape:", College.shape)
# (a)
display(College.head())
display(College.describe())
```

✓ 0.0s

Dataset shape: (777, 18)

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	FUndergrad	PUndergrad	Outstate	Room.Board	Books	Personal	PhD	Terminal	S.F.Ratio	perc.alumni	Expend	Grad.Rate
0	Yes	1660	1232	721	23	52	2885	537	7440	3300	450	2200	70	70	18.1	12	7041	60
1	Yes	2186	1924	512	16	29	2683	1227	12280	6450	750	1500	29	30	12.2	16	10527	56
2	Yes	1428	1097	336	22	50	1036	99	11250	3750	400	1165	53	66	12.9	30	8735	54
3	Yes	417	349	137	60	89	510	63	12960	5450	450	875	92	97	7.7	37	19016	59
4	Yes	193	146	55	16	44	249	869	7560	4120	800	1500	76	72	11.9	2	10922	15

	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad	Outstate	Room.Board	Books
count	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000
mean	3001.638353	2018.804376	779.972973	27.558559	55.796654	3699.907336	855.298584	10440.669241	4357.526384	549.380952
std	3870.201484	2451.113971	929.176190	17.640364	19.804778	4850.420531	1522.431887	4023.016484	1096.696416	165.105360
min	81.000000	72.000000	35.000000	1.000000	9.000000	139.000000	1.000000	2340.000000	1780.000000	96.000000
25%	776.000000	604.000000	242.000000	15.000000	41.000000	992.000000	95.000000	7320.000000	3597.000000	470.000000
50%	1558.000000	1110.000000	434.000000	23.000000	54.000000	1707.000000	353.000000	9990.000000	4200.000000	500.000000
75%	3624.000000	2424.000000	902.000000	35.000000	69.000000	4005.000000	967.000000	12925.000000	5050.000000	600.000000
max	48094.000000	26330.000000	6392.000000	96.000000	100.000000	31643.000000	21836.000000	21700.000000	8124.000000	2340.000000

Personal	PhD	Terminal	S.F.Ratio	perc.alumni	Expend	Grad.Rate
777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000
1340.642214	72.660232	79.702703	14.089704	22.743887	9660.171171	65.46332
677.071454	16.328155	14.722359	3.958349	12.391801	5221.768440	17.17771
250.000000	8.000000	24.000000	2.500000	0.000000	3186.000000	10.00000
850.000000	62.000000	71.000000	11.500000	13.000000	6751.000000	53.00000
1200.000000	75.000000	82.000000	13.600000	21.000000	8377.000000	65.00000
1700.000000	85.000000	92.000000	16.500000	31.000000	10830.000000	78.00000
6800.000000	103.000000	100.000000	39.800000	64.000000	56233.000000	118.00000

b/	<pre> # (b) n_private = (College["Private"] == "Yes").sum() print(f"Number of private colleges: {n_private}") </pre> ✓ 0.0s Number of private colleges: 565
----	---

c/	<pre> # (c) College["Elite"] = "No" College.loc[College["Top10perc"] > 50, "Elite"] = "Yes" n_elite = (College["Elite"] == "Yes").sum() print(f"The number of Elite colleges is: {n_elite}") </pre> ✓ 0.0s The number of Elite colleges is: 78
----	---

d/

```
# (d)
import matplotlib.pyplot as plt
from matplotlib.pyplot import subplots
fig, axes = subplots(1, 2, figsize=(12, 4))

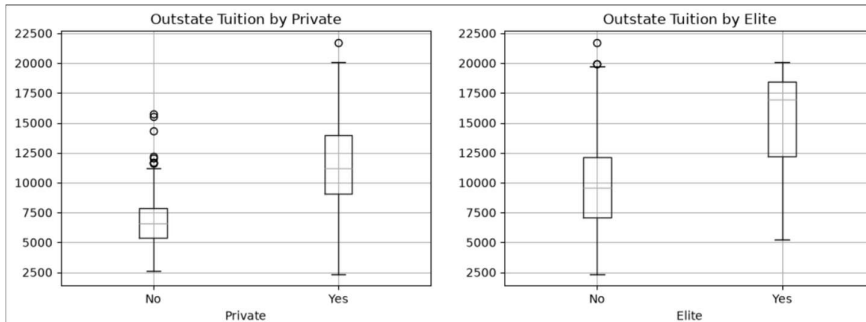
College.boxplot("Outstate", by="Private", ax=axes[0])
axes[0].set_title("Outstate Tuition by Private")
axes[0].set_xlabel("Private")

College.boxplot("Outstate", by="Elite", ax=axes[1])
axes[1].set_title("Outstate Tuition by Elite")
axes[1].set_xlabel("Elite")

fig.suptitle("")

✓ 0.3s

Text(0.5, 0.98, '')
```



e/

```
# (e)
import matplotlib.pyplot as plt
from matplotlib.pyplot import subplots
fig, axes = subplots(2, 2, figsize=(10, 8))
# four quantitative variables with different numbers of bins
College.hist("Apps", bins=15, ax=axes[0, 0])
axes[0, 0].set_title("Applications (bins=15)")

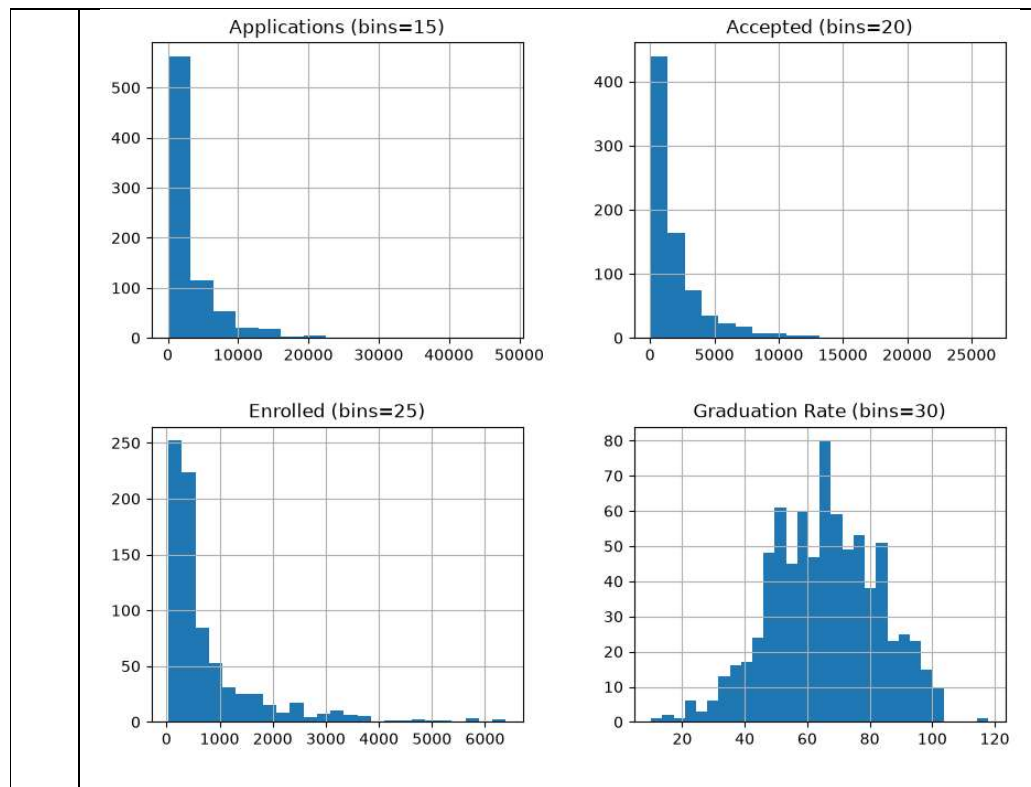
College.hist("Accept", bins=20, ax=axes[0, 1])
axes[0, 1].set_title("Accepted (bins=20)")

College.hist("Enroll", bins=25, ax=axes[1, 0])
axes[1, 0].set_title("Enrolled (bins=25)")

College.hist("Grad.Rate", bins=30, ax=axes[1, 1])
axes[1, 1].set_title("Graduation Rate (bins=30)")

✓ 0.7s

Text(0.5, 1.0, 'Graduation Rate (bins=30)')
```

Exercise 7 - Loops and string formatting

1. *Task Requirement:* Automate the reporting of statistical properties for multiple columns using a for loop, embedding the results into precisely aligned text strings.
2. *Concepts Used:* for loops, boolean averaging, f-string formatting (`[:<12, :.2f, :.1%]`).
3. *Actual Code & Output*

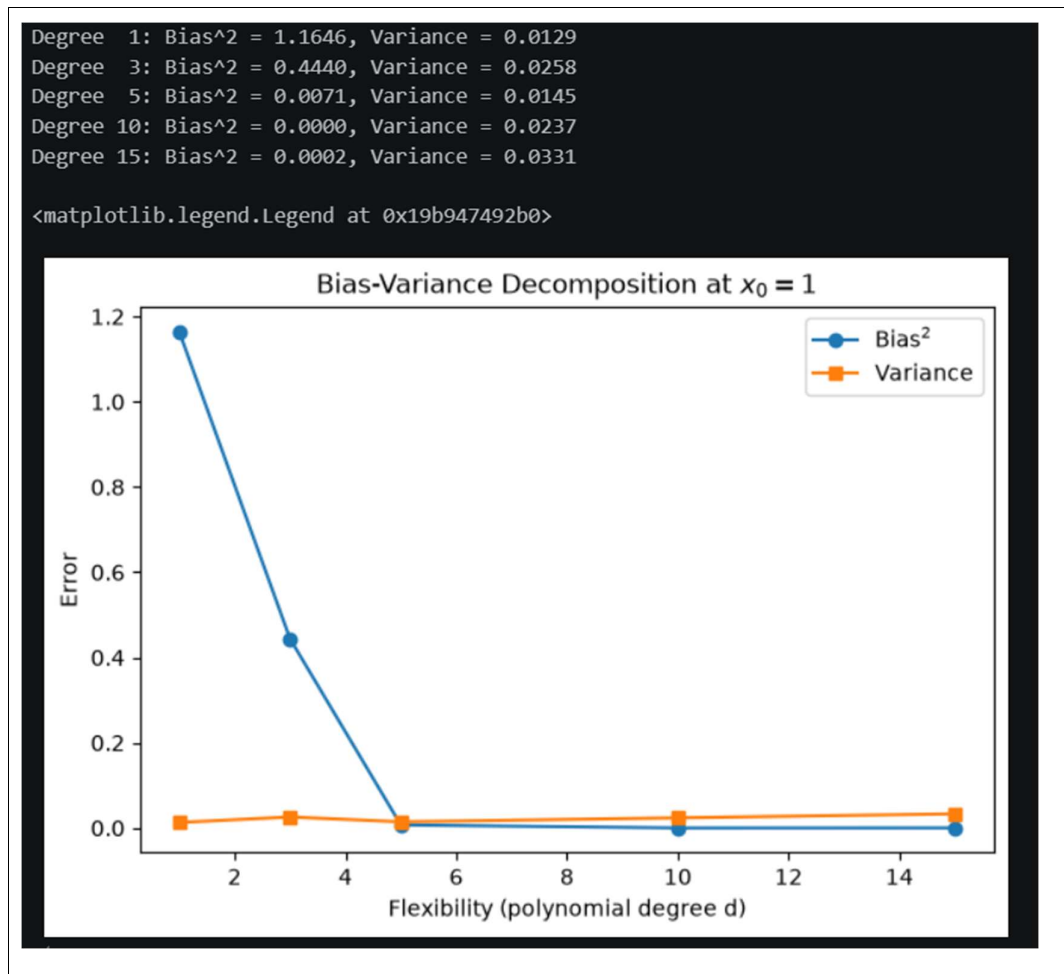
```
quantitative = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration"]

for col in quantitative:
    col_mean = Auto_clean[col].mean()
    col_sd = Auto_clean[col].std()
    above_mean_share = (Auto_clean[col] > col_mean).mean()
    print(f"{col:<12} mean = {col_mean:.2f} sd = {col_sd:.2f} above-mean share = {above_mean_share:.1%}")

✓ 0.0s
acceleration mean = 15.54 sd = 2.76 above-mean share = 45.7%
```

Exercise 8 - Estimating bias and variance by simulation

1. *Task Requirement:* Empirically validate the Bias-Variance trade-off theory by simulating 200 polynomial regressions across multiple flexibility degrees (d) at a fixed test point $x_0 = 1$.
2. *Concepts Used:* Monte Carlo simulation loops, `np.polyfit`, `np.polyval`, Bias/Variance mathematical formulation.
3. *Actual Code & Output*



Exercise 9 - Choosing K for KNN

1. *Task Requirement:* Calculate and plot Training vs. Test error rates for a K-Nearest-Neighbours classifier across $K \in [1, 100]$ to identify the optimal complexity sweet spot.

2. *Concepts Used*: `knn_predict()`, boolean mismatch averaging (`!=`), logarithmic scaling (`ax.set_xscale('log')`).

3. *Actual Code & Output*



PART 5: SCHOLAR DISCUSSION

This lab establishes the programming paradigms necessary for applied statistical modeling.

- Data Structures & Cleaning: Mastery of pandas indexing (`.loc`, `.iloc`) and NaN handling is a prerequisite for feeding clean tensors into machine learning algorithms.
- Vectorization: We observed that utilizing mathematical formulas directly on arrays (e.g., computing boolean means) bypasses traditional loops, operating at C-level speeds underneath the Python wrapper.
- Empirical Theory Validation: Challenge Exercises 8 and 9 showcased that complex theoretical theorems (like the Bias-Variance Decomposition) can be definitively proven via brute-force computational simulation (Monte Carlo), bridging abstract math with applied computer science.

PART 6: FINAL REFLECTION

Lab 1 successfully transitioned my skill set from basic programming mechanics into the domain of data science. The most profound realization was observing the Bias-Variance trade-off physically manifest in Exercise 8 and 9. It proved that in statistical learning, giving an algorithm more flexibility (higher polynomials or lower K) is not inherently better; it is a delicate balance of minimizing structural bias without suffocating the model in variance. The robust data-wrangling techniques developed here using NumPy and pandas will serve as the foundational infrastructure for deploying more advanced models in upcoming labs.